Task 2: Context-Aware NL2SPARQL with Interactive Linking

Goal

Build a chatbot-style service that converts natural language questions into SPARQL for a target knowledge graph, with a configurable LLM backend and an interactive, validated entity-linking loop. The system must accept graph context (schemas, example queries, templates) and assure query correctness via validation and user-in-the-loop confirmations.

Motivation

NL2SPARQL systems often hallucinate properties, pick wrong entity identifiers, or support only narrow question types. By allowing to load graph context (RDFS/OWL, ShEx/SHACL, prefixes, templates/examples) and adding an entity-linking approval loop plus post-generation checks, this task aims for a reusable, configurable pipeline that is safer and more accurate. Choose your domain focus (e.g., DBLP with CEUR-WS data or Wikidata with Scholia context), making the solution adaptable to real-world scholarly KGs.

Core requirements

Pipeline

- modular stages for mention extraction,
- candidate entity linking, user disambiguation, prompt construction with context, LLM generation, static/semantic validation, execution, and result display.

Model-agnostic LLM

- integrate via an Ollama adapter;
- support switching among at least two local models;
- no hard dependency on any single model.

Context ingestion

- load “context packs” (prefixes, schema, ShEx/SHACL if available, example queries, templates) selectable per deployment/domain.
- explore other ways of providing context information for increasing the accuracy

Interactive entity linking

- extract mentions, rank candidates, get user approval, and constrain generation with approved IRIs;
- re-check entities used in the final query.

SPARQL safety and validation

- syntax, prefix resolution, existence checks for properties/classes;
- enforce a configurable allowed feature subset, decide on relevant features to ensure reliability.

Deliverables

- documented, containerized deployment;
- configuration for models, endpoints, context packs;
- language is group's choice (Python + RDFLib recommended)
- minimal web UI (chat, entity approval, SPARQL preview/edit, model/context switch); aesthetics not the focus.
- tested and documented code

Acceptance Criteria

Models and architecture	
AC-1	Support at least two Ollama-hosted models (e.g., Llama 3, Mistral/Qwen) behind a single adapter; switching requires no code change.
AC-2	Active model can be changed via config/UI and takes effect on the next generation request (≤1 minute)
AC-3	Clear module boundaries for: mention extraction, entity linking, prompt builder, generator, validator, executor; interfaces documented.
AC-4	Configuration files for setup define model(s), endpoint, context pack, and allowed SPARQL subset; reload via restart or hot-reload. Optional over the web interface.
AC-5	Caches schema/context and entity lookups to avoid repeated network calls.

KG context and data	
AC-6	Loads context data at startup or via admin UI containing prefixes, schema (RDFS/OWL), and at least 10 example queries and/or templates.
AC-7	Context data influences generation (properties/classes from the pack are preferred and referenced).

KG context and data	
AC-8	Choose a KG focus (DBLP with CEUR-WS or Wikidata/Scholia) and provide a working SPARQL endpoint configuration.
AC-9	Context data are swappable without code changes; Context data is viewable in the UI.
AC-10	If DBLP is chosen: pack includes IRIs for core scholarly concepts (author, publication, venue/workshop, year) and at least 10 DBLP-specific example queries

Entity linking assurance loop	
AC-11	Devise feedback loop to fix possible errors in IRI Entity Linking.
AC-12	Implements strategies for detection and adjustment of incorrect NEL results or used IRIs in the queries.
AC-13	Auto-approval can be applied when a confidence/score threshold is met; users can override any auto choice.
AC-14	All linking decisions (candidates, scores, approvals) are logged alongside the final query.

SPARQL generation, validation, and safety	
AC-15	Generates syntactically valid SPARQL for at least SELECT queries; ASK optional.
AC-16	Given a SPARQL query (text or .rq), returns “Valid SPARQL query” or a specific error within 3 seconds.
AC-17	Validates that prefixes resolve and that properties/classes in the query exist (via schema/context or endpoint introspection); reports actionable errors.
AC-18	Enforces a configurable allowed SPARQL subset (forbidden features list decided in phase one); blocked features are reported to the user.
AC-19	Executes queries against the configured endpoint and displays results; execution errors are caught and shown with guidance.

UI, evaluation, and delivery	
AC-20	Web UI supports: question input, entity approval, SPARQL preview/edit, model switch, and context selection; results shown in a table with downloadable JSON.
AC-21	Define an evaluation set in phase one with ≥5 distinct query types, each with multiple paraphrases; include expected answers (query or result pattern).
AC-22	Provide an evaluation script reporting at least: syntax validity rate, entity linking precision, execution success rate, and answer correctness.
AC-23	Logging/tracing includes a request/trace ID and records each pipeline step
AC-24	Reproducible delivery: README, architecture diagram, config schema, and Docker-based deployment; include unit tests for validation and one end-to-end test.

Additional Material

Existing Approaches

- ASK-DBLP: Answering Questions over DBLP
  - ASK-DBLP enables users to ask questions in natural language and automatically converts them into SPARQL queries to provide precise answers. The platform is designed to be robust and user-friendly, allowing refinement of ambiguous queries and selection or updating of entity links within the SPARQL query. This approach makes ASK-DBLP adaptable to the latest DBLP schema updates.
  - Try the demo: https://ask-dblp.nliwod.org
- DBLPLink 2.0 – An Entity Linker for the DBLP Scholarly Knowledge Graph
  - DBLPLink 2.0 introduces a novel zero-shot entity linking approach using LLMs. Candidate entities are re-ranked based on the log-probabilities of the 'yes' token in the LLM's penultimate layer, resulting in more accurate and adaptive entity linking for the DBLP Knowledge Graph.
  - Try the demo: https://ask-dblp.nliwod.org

References

- Sorry, i don't speak SPARQL: translating SPARQL queries into natural language
- First International TEXT2SPARQL Challenge

Standards

- https://www.w3.org/TR/sparql11-query/
- https://www.w3.org/TR/rdf11-concepts/

Next Steps

In the first phase your group will conduct a requirements analysis for the implementation of the given task. The analysis should identify suitable libraries and technologies (rdf frameworks, SPARQL processors, LLM frameworks), examine existing solutions in the field, and define a comprehensive set of functional and non-functional requirements that specify what needs to be built. Decide on a use case focus for the task and evaluate the corresponding knowledge graph.

Your deliverables for the midterm presentation are:

1. a written requirements analysis report covering related work, technical architecture recommendations, and detailed requirements specification, and
2. a presentation summarizing your key findings, proposed technology stack, software architecture and implementation roadmap.